

DIGITAL LOGIC DESIGN

Lecture 10

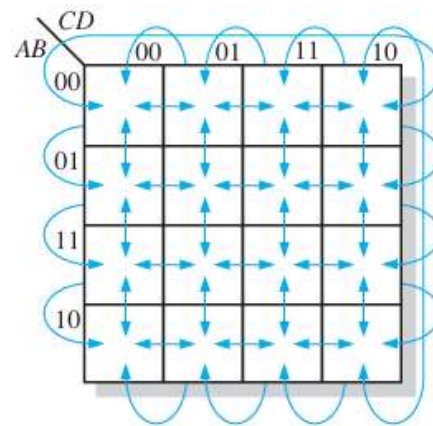


FIGURE 4-27 Adjacent cells on a Karnaugh map are those that differ by only one variable. Arrows point between adjacent cells.

Steps to Solve Expression using K-map

1. Select the K-map according to the number of variables.
2. Identify minterms or maxterms as given in the problem.
3. For SOP put 1's in blocks of K-map respective to the minterms (0's elsewhere).
4. For POS put 0's in blocks of K-map respective to the max terms (1's elsewhere).
5. Make rectangular groups containing total terms in power of two like 2,4,8 ..(except 1) and try to cover as many elements as you can in one group.
6. From the groups made in step 5 find the product terms and sum them up for SOP form.

Advantages of K-MAP

- **Makes Logic Simpler:** It makes complicated Boolean expressions simpler.
- **Minimizes Logic Gates:** Simplifying the logic helps us to use fewer logic gates, making circuits more efficient.
- **Reduce Errors:** The visual representation of k-map helps to avoid errors while simplifying.
- **Time-Saving:** It's quicker than traditional methods for simplifying logic.
- **No need for Boolean Laws:** K-map doesn't require deep knowledge of Boolean laws, making it easy for beginners.

Disadvantages of K-MAP

- **Limited to Fewer Variables:** K-maps are best suited for 2 to 4 variables and above it, process becomes hard and complicated to manage.
- **Not suitable for all functions:** In some cases, its hard to group terms correctly, leading to errors and making simplification difficult.
- **Space Limitations:** As the number of variables increases, the K-map grid becomes too large to handle easily.
- **Requires Careful Grouping:** Sometimes incorrect grouping of terms can cause mistakes in logic simplification.

Example:

Group the 1s in each of the Karnaugh maps in Fig.(5-6).

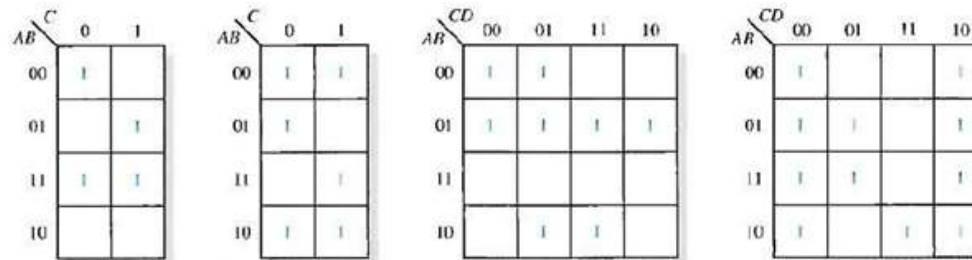


Fig.(5-6)

Solution:

The groupings are shown in Fig.(5-7). In some cases, there may be more than one way to group the 1s to form maximum groupings.

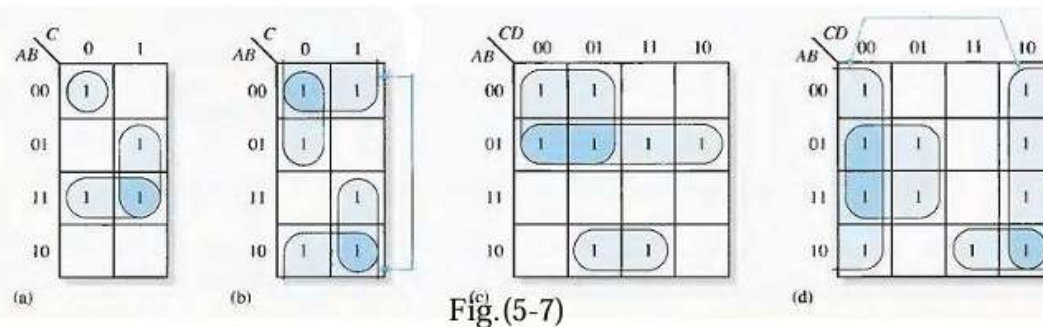
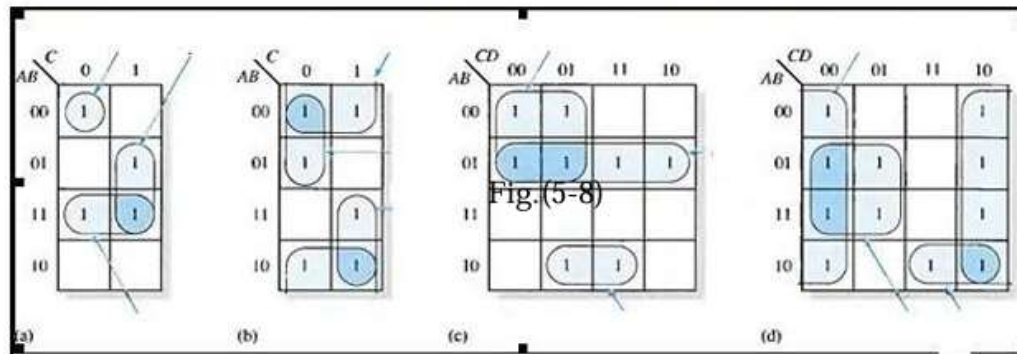


Fig.(5-7)

Example:

Determine the product terms for each of the Karnaugh maps in Fig.(5-7) and write the resulting minimum SOP expression.



Solution:

The resulting minimum product term for each group is shown in Fig.(5-8). The minimum SOP expressions for each of the Karnaugh maps in the figure are:

(a) $AB + BC + \overline{A}BC$

(b) $\overline{B} + AC + \overline{A}\overline{C}$

(c) $\overline{A}B + \overline{A}C + ABD$

(d) $\overline{D} + \overline{A}BC + B\overline{C}$

Example: Use a Karnaugh map to minimize the following standard SOP expression:

$$ABC + ABC + ABC + ABC + ABC$$

Example: Use a Karnaugh map to minimize the following SOP expression:

$$\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}\overline{C}D + \overline{A}\overline{B}C\overline{D} + \overline{A}\overline{B}CD + \overline{A}B\overline{C}\overline{D} + \overline{A}B\overline{C}D + \overline{A}BC\overline{D} + \overline{A}BCD$$

Figure 8-37

KARNAUGH MAP POS MINIMIZATION

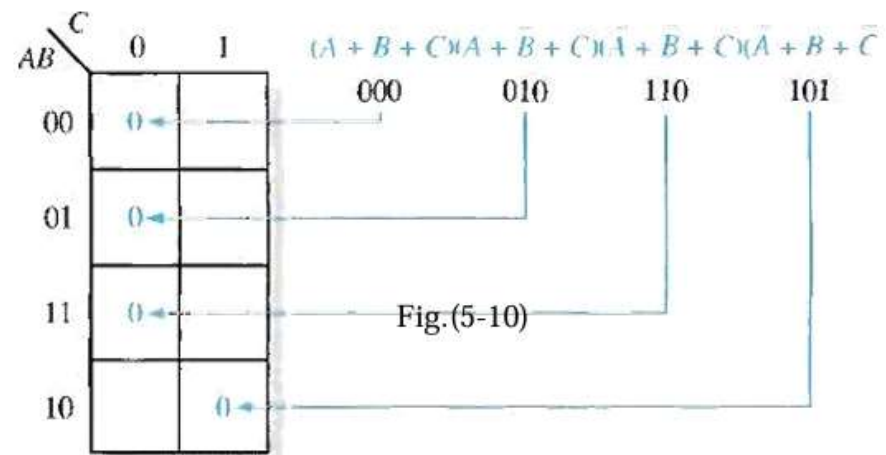
In this section, we will focus on POS expressions. The approaches are much the same except that with POS expressions, 0s representing the standard sum terms are placed on the Karnaugh map instead of 1s.

For a POS expression in standard form, a 0 is placed on the Karnaugh map for each sum term in the expression. Each 0 is placed in a cell corresponding to the value of a sum term. For example, for the sum term $A + B + C$, a 0 goes in the 0 1 0 cell

When a POS expression is completely mapped, there will be a number of 0s on the Karnaugh map equal to the number of sum terms in the standard POS expression. The cells that do not have a 0 are the cells for which the expression is 1. Usually, when working with POS expressions, the 1s are left off. The following steps and the illustration in Fig.(5-10) show the mapping process.

Step 1. Determine the binary value of each sum term in the standard POS expression. This is the binary value that makes the term equal to 0.

Step 2. As each sum term is evaluated, place a 0 on the Karnaugh map in the corresponding cell.



Example of mapping a standard POS expression.

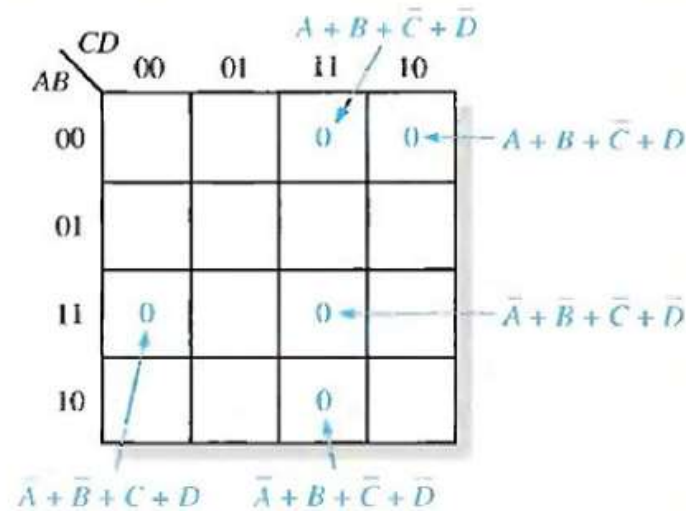
Example:

Map the following standard POS expression on a Karnaugh map:

$$(\bar{A} + \bar{B} + C + D)(\bar{A} + B + \bar{C} + \bar{D})(A + B + \bar{C} + D)(\bar{A} + \bar{B} + \bar{C} + \bar{D})(A + B + \bar{C} + \bar{D})$$

Solution:

$$\begin{array}{ccccc} (\bar{A} + \bar{B} + C + D) & (\bar{A} + B + \bar{C} + \bar{D}) & (A + B + \bar{C} + D) & (\bar{A} + \bar{B} + \bar{C} + \bar{D}) & (A + B + \bar{C} + \bar{D}) \\ 1100 & 1011 & 0010 & 1111 & 0011 \end{array}$$



Karnaugh Map Simplification of POS Expressions

The process for minimizing a POS expression is basically the same as for an SOP expression except that you group 0s to produce minimum sum terms instead of grouping 1s to produce minimum product terms. The rules for grouping the 0s are the same as those for grouping the 1s that you learned before.

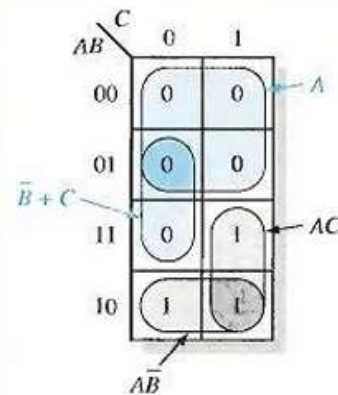
Example:

Use a Karnaugh map to minimize the following standard POS expression:

Also, derive the equivalent SOP expression.

$$(A + B + C)(A + B + \bar{C})(A + \bar{B} + C)(A + \bar{B} + \bar{C})(\bar{A} + \bar{B} + C)$$

Solution:



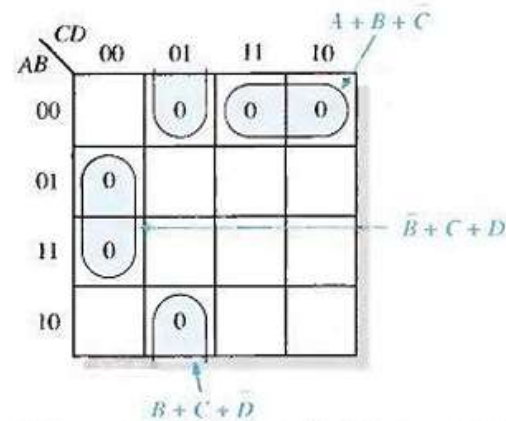
Example: Use a Karnaugh map to minimize the following POS expression:

$$(B + C + D)(A + B + \bar{C} + D)(\bar{A} + B + C + \bar{D})(A + \bar{B} + C + D)(\bar{A} + \bar{B} + C + D)$$

Example: Using a Karnaugh map, convert the following standard POS expression into a minimum POS expression, a standard SOP expression, and

$$(\bar{A} + \bar{B} + C + D)(A + \bar{B} + C + D)(A + B + C + \bar{D})$$

$$(A + B + \bar{C} + \bar{D})(\bar{A} + B + C + \bar{D})(A + B + \bar{C} + D)$$



(a) Minimum POS: $(A + B + C)(\bar{B} + C + D)(B + C + \bar{D})$

		$\overline{A}BCD$		$A\overline{B}CD$		
AB	CD	00	01	11	10	
00		1	0	0	0	$\overline{A}BCD$
01		0	1	1	1	$\overline{A}BCD$
11		0	1	1	1	$ABCD$
10		1	0	1	1	$ABCD$
		$\overline{A}BCD$	$\overline{A}BCD$	$ABCD$	$ABCD$	

(b) Standard SOP:

$$\overline{A}BCD + \overline{A}BCD + \overline{A}BCD + \overline{A}BCD + ABCD + ABCD + ABCD + ABCD + ABCD + ABCD + ABCD + ABCD$$

"Don't Care" Conditions

Sometimes a situation arises in which some input variable combinations are not allowed. For example, recall that in the BCD code there are six invalid combinations: 1010, 1011, 1100, 1101, 1110, and 1111. Since these unallowed states will never occur in an application involving the BCD code, they can be treated as "don't care" terms with respect to their effect on the output. That is, for these "don't care" terms either a 1 or a 0 may be assigned to the output: it really does not matter since they will never occur.

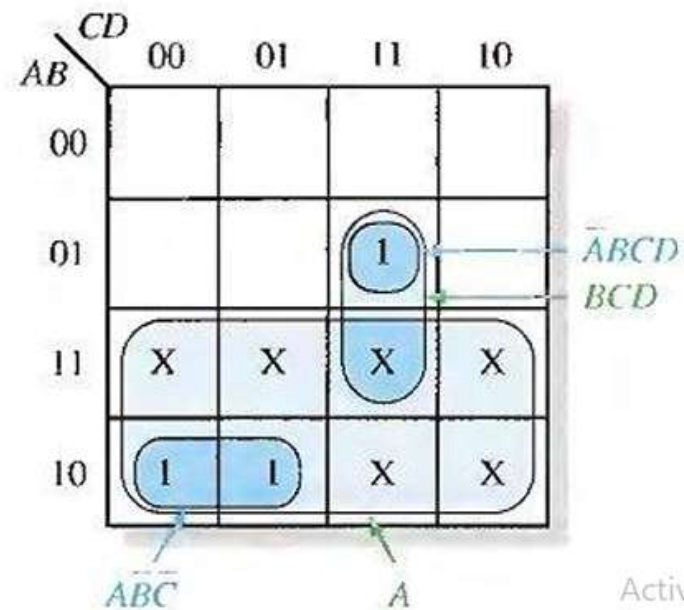
The "don't care" terms can be used to advantage on the Karnaugh map. Fig.(5-9) shows that for each "don't care" term, an X is placed in the cell. When grouping the 1s, the Xs can be treated as 1s to make a larger grouping or as 0s if they cannot be used to advantage. The larger a group, the simpler the resulting term will be.

The truth table in Fig.(5-9)(a) describes a logic function that has a 1 output only when the BCD code for 7,8, or 9 is present on the inputs. If the "don't cares" are used as 1s, the resulting expression for the function is $A + BCD$, as indicated in part (b). If the "don't cares" are not used as 1s, the resulting expression is $ABC + ABCD$: so you can see the advantage of using "don't care" terms to get the simplest expression.

<i>ABCD</i>	<i>Y</i>
0 0 0 0	0
0 0 0 1	0
0 0 1 0	0
0 0 1 1	0
0 1 0 0	0
0 1 0 1	0
0 1 1 0	0
0 1 1 1	1
1 0 0 0	1
1 0 0 1	1
1 0 1 0	X
1 0 1 1	X
1 1 0 0	X
1 1 0 1	X
1 1 1 0	X
1 1 1 1	X

(a) Truth table

Don't cares

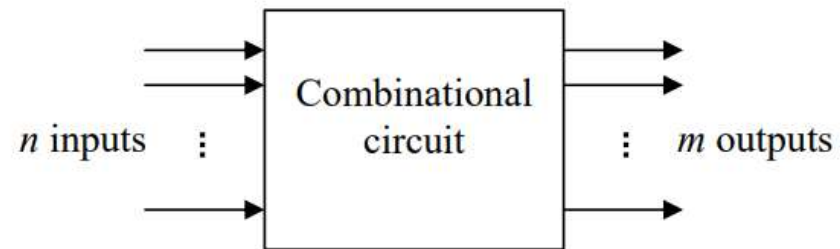


(b) Without "don't cares" $Y = \overline{A}BCD + ABC$

Activate
Go to Setting

COMBINATIONAL LOGIC DESIGN

- A combinational circuit consists of input variables, output variables, and logic gates that transform binary information from the input to the output.



Analysis and design procedures

The following are the basic steps to design a combinational circuits

1. Define the problem.
2. Determine the number of input and output variables.
3. Fix a letter symbols to the input and the outputs. (eg. A,B,C ,w, x, Y,F, etc)
4. Get the relationship between input and output from the truth table.
5. By using K-map obtain the simplified Boolean expression for the outputs.
6. Draw the logic diagram using gates.

2.2 Adder

The Basic operation in digital computer is binary addition. The circuit which perform the addition of binary bits are called as Adder.

The logic circuit which perform the addition of two bit is called Half adder and three bit is called Full adder.

Rules for two bit addition

$0 + 0 = 0$
$0 + 1 = 1$
$1 + 0 = 1$
$1 + 1 = 10_2$

2.2.1 Half Adder

The two inputs of the half adders are augend and addend, the outputs are sum and carry.

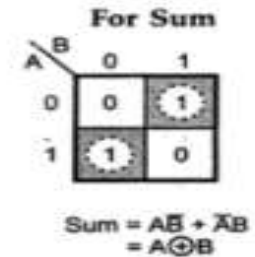
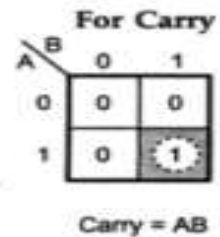
Block diagram of Half adder



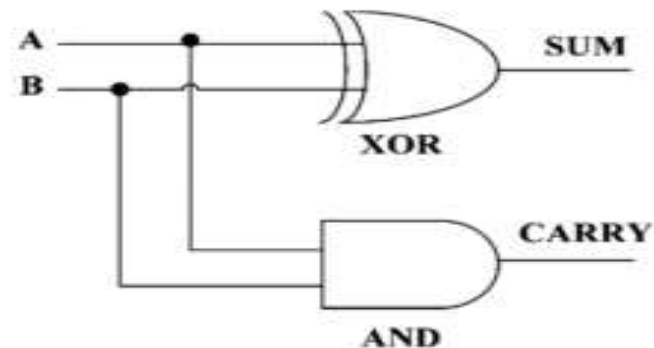
Truth table of Half adder

Inputs		Outputs	
A	B	Carry	Sum
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

K-map simplification

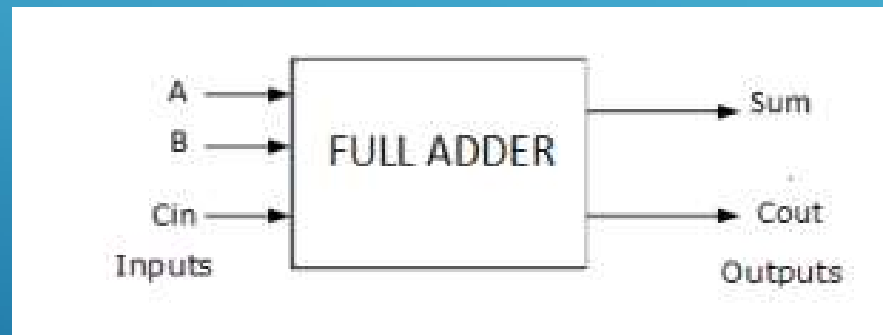


Logic diagram



2.2.2 Full Adder

The three inputs of the full adders are augend, addend and the carry input from the previous addition, the outputs are sum and carry



Truth table

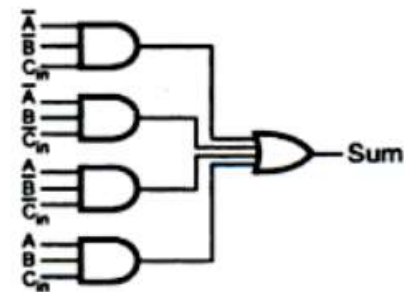
Inputs			Outputs	
A	B	Cin	Cout	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

K-map simplifications

For Sum

		AB			
C _{in}		00	01	11	10
		0	1		1
		1	1	1	

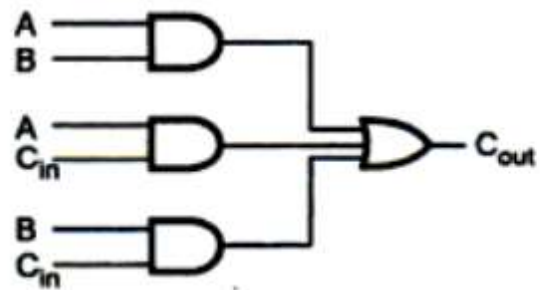
$$\text{Sum} = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in}$$



For Cout

C_{in} \ AB	AB			
	00	01	11	10
0			1	
1		1	1	1

$$C_{out} = AB + BC_{in} + AC_{in}$$



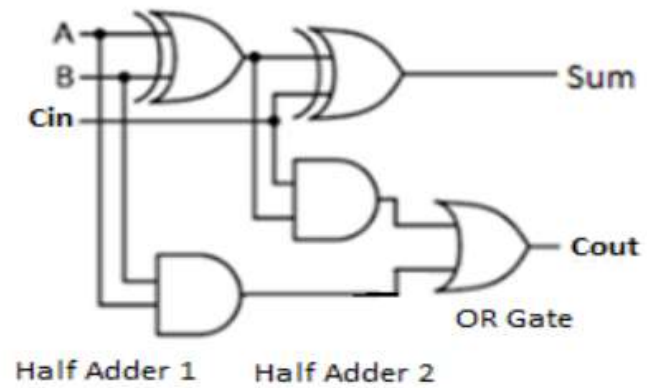
The expression for sum is

$$\begin{aligned}\text{Sum} &= \bar{A} \bar{B} C_{in} + \bar{A} B \bar{C}_{in} + A \bar{B} \bar{C}_{in} + A B C_{in} \\ &= C_{in} (\bar{A} \bar{B} + AB) + \bar{C}_{in} (\bar{A} B + A \bar{B}) \\ &= C_{in} (A \cdot B) + \bar{C}_{in} (A \oplus B) \\ &= C_{in} (\overline{A \oplus B}) + \bar{C}_{in} (A \oplus B) \\ &= C_{in} \oplus (A \oplus B)\end{aligned}$$

The Expression for carry is

$$\begin{aligned}C_{out} &= AB + A C_{in} + BC_{in} \\&= AB + A C_{in} + BC_{in} (A + \bar{A}) \\&= ABC_{in} + AB + A C_{in} + \bar{A} BC_{in} \\&= AB (C_{in} + 1) + A C_{in} + \bar{A} BC_{in} \\&= AB + AC_{in} + \bar{A} BC_{in} \\&= AB + A C_{in} (B + \bar{B}) + \bar{A} B C_{in} \\&= AB C_{in} + AB + A \bar{B} C_{in} + \bar{A} B C_{in} \\&= AB (C_{in} + 1) + A \bar{B} C_{in} + \bar{A} B C_{in} \\&= AB + A \bar{B} C_{in} + \bar{A} B C_{in} \\&= AB + C_{in} (A \bar{B} + \bar{A} B) \\&= AB + C_{in} (A \oplus B)\end{aligned}$$

Logic Diagram



2.3 Subtractor

Subtractor is the logic circuit which is used to subtract two binary number (digit) and provides Difference and Borrow as a output. In digital electronics we have two types of subtractor, Half Subtractor and Full Subtractor.

$0 - 0 = 0$
$0 - 1 = 1 \text{ with borrow } 1$
$1 - 0 = 1$
$1 - 1 = 0$

2.3.1 Half Subtractor

Half Subtractor is used for subtracting one single bit binary digit from another single bit binary digit. The truth table of Half Subtractor is shown below.

K-map for Difference and Borrow

For Difference

A \ B	0	1
0	0	1
1	1	0

$$\begin{aligned}\text{Difference} &= \overline{A}B + A\overline{B} \\ &= A \oplus B\end{aligned}$$

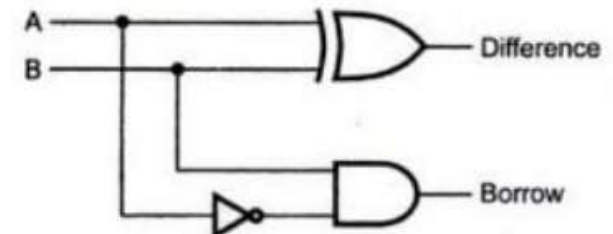
For Borrow

A \ B	0	1
0	0	1
1	0	0

$$\text{Borrow} = \overline{A}B$$

Inputs		Outputs	
A	B	Difference	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Logic Diagram



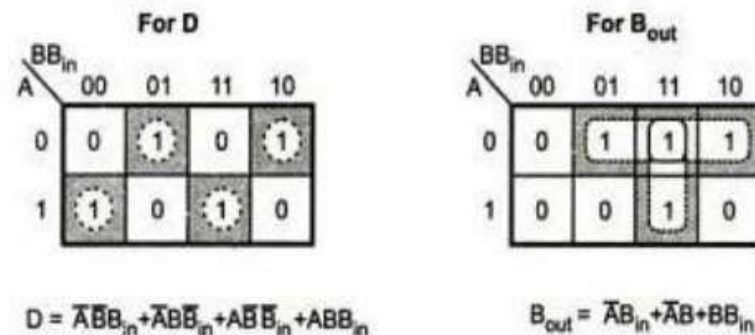
2.3.2 Full Subtractor

A logic Circuit Which is used for Subtracting Three Single bit Binary digit is known as Full Subtractor. The inputs are A, B, Bin and the outputs are D and Bout.

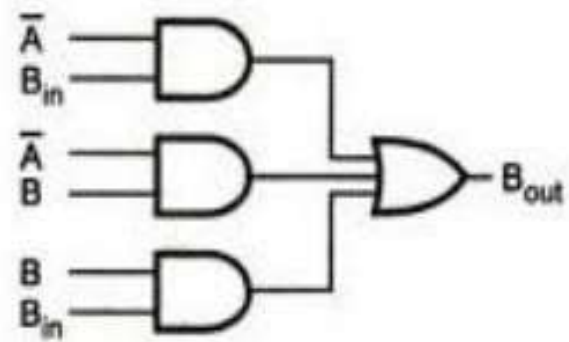
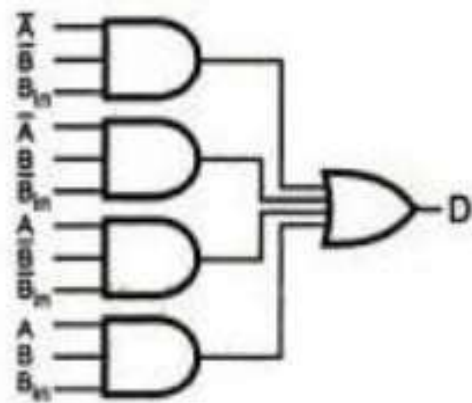
Truth table

Inputs			Outputs	
A	B	Bin	D	Bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

K-map for D and Bout



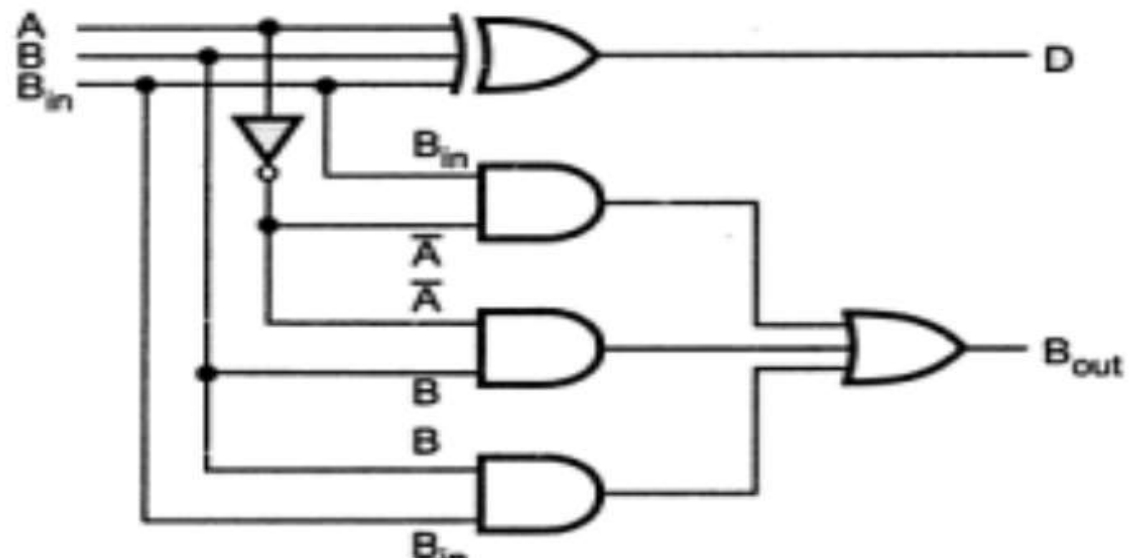
Logic Diagram



We can further simplify the function of the Difference (D)

$$\begin{aligned}
 D &= \bar{A} \bar{B} B_{in} + \bar{A} B \bar{B}_{in} + A \bar{B} \bar{B}_{in} + A B B_{in} \\
 &= B_{in} (\bar{A} \bar{B} + AB) + \bar{B}_{in} (\bar{A} B + A \bar{B}) \\
 &= B_{in} (A \odot B) + \bar{B}_{in} (A \oplus B) \\
 &= B_{in} (\overline{A \oplus B}) + \bar{B}_{in} (A \oplus B) \\
 &= B_{in} \oplus (A \oplus B)
 \end{aligned}$$

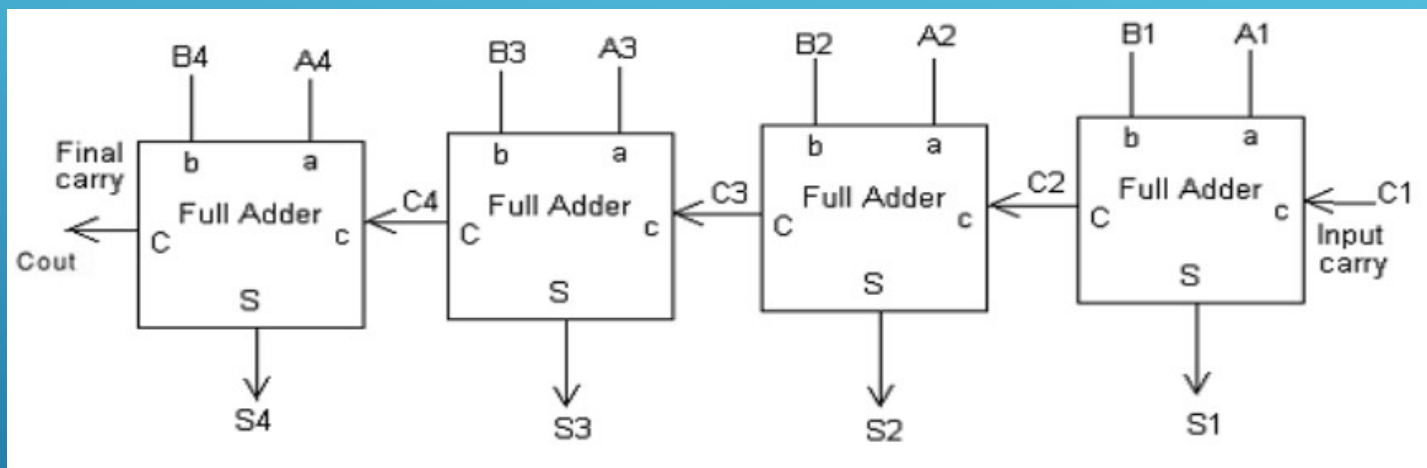
Simplified Logic diagram



2.4 Parallel Adder – Subtractor

2.4.1 Four bit Parallel binary Adder

In practical situations it is required to add two data each containing more than one bit. Two binary numbers each of n bits can be added by means of a full adder circuit. Consider the example that two 4-bit binary numbers $B_4B_3B_2B_1$ and $A_4A_3A_2A_1$ are to be added with a carry input C_1 . This can be done by cascading four full adder circuits. The least significant bits A_1 , B_1 , and C_1 are added to produce sum output S_1 and carry output C_2 . Carry output C_2 is then added to the next significant bits A_2 and B_2 producing sum output S_2 and carry output C_3 . C_3 is then added to A_3 and B_3 and so on. Thus finally producing the four-bit sum output $S_4S_3S_2S_1$ and final carry output C_{out} .



2.4.2 Four Bit Parallel Binary Subtractor

We can design a four bit parallel subtractor by connecting three full subtractors and one half subtractor. In the figure $A = A_3 A_2 A_1 A_0$ is minuend $B = B_3 B_2 B_1 B_0$ is subtrahend giving the difference $D = D_3 D_2 D_1 D_0$.

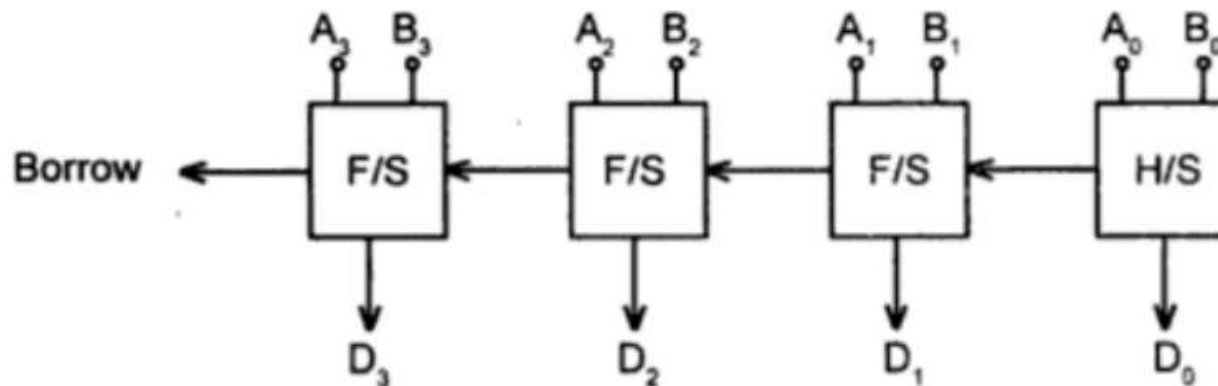


Fig. Block diagram of 4 bit binary parallel Subtractor